



Shailesh Yadav-da
Data Analyst

NumPy— Complete Course to
Transform Your Data Skills

Welcome To My Post

Scroll More >

1. What is NumPy?

NumPy (Numerical Python) is a library used for:

- Fast numerical computation
- Working with arrays (better than Python lists)
- Foundation for libraries like Pandas, Matplotlib, Seaborn

In simple terms:

NumPy = High-speed math + powerful arrays

2. Why NumPy is Important

Normal Python lists are:

- Slow
- Memory inefficient

NumPy arrays are:

- Fast (written in C internally)
- Memory efficient
- Support vectorized operations

3. Install & Import

`pip install numpy`

`import numpy as np`

4. Core Concept → ndarray

Everything in NumPy revolves around:

ndarray (N-dimensional array)

5. Creating Arrays

From list

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4])  
print(arr)
```

NumPy Array vs Python List

Let's **compare** them in a data analyst perspective (not theory overload).

◆ 1. Basic Difference

Feature	Python List	NumPy Array
Type	General purpose	Numerical computing
Speed	Slow ❌	Fast ✅
Memory	High ❌	Low ✅
Operations	Manual (loops) ❌	Vectorized ✅

2. Performance (REAL GAME CHANGER)

Python List (Slow)

```
lst = [1, 2, 3, 4]

result = []
for i in lst:
    result.append(i * 2)

print(result)
```

NumPy Array (Fast)

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr * 2)
```

Why fast?

NumPy uses C-level implementation + vectorization

How to Create NumPy Array

1. From Python List (Most Used)

```
arr = np.array([1, 2, 3, 4])  
print(arr)
```

Used in 90% real cases

2. 2D Array (Matrix Form)

```
arr = np.array([[1, 2], [3, 4]])  
print(arr)
```

Used for:

- Tables
- Dataset structure

3. Using arange() (Range Data)

```
arr = np.arange(0, 10, 2)  
print(arr)
```

Output: [0 2 4 6 8]

Best for sequence data

4. Using linspace() (Equal Spacing)

```
arr = np.linspace(0, 1, 5)  
print(arr)
```

Output: [0. 0.25 0.5 0.75 1.]

Used in:

- Graphs
- Data distribution

5. Zeros Array

```
arr = np.zeros((2, 3))  
print(arr)
```

Output:

```
[[0. 0. 0.]  
 [0. 0. 0.]]
```

Use:

- Initialize empty data

6. Ones Array

```
arr = np.ones((2, 3))  
print(arr)
```

Use:

- Default values / testing

7. Random Values (Important)25

```
arr = np.random.rand(2, 3)  
print(arr)
```

Use:

- Sample data
- ML / testing

8. Empty Array

```
arr = np.empty((2,3))  
print(arr)
```

Output: random garbage values (not zero)

Why?

- Memory allocated but not initialized

Use Case:

- When you will fill values later
- Faster than zeros()

9. Diagonal Array

```
arr = np.eye(3)  
print(arr)
```

Output:

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

Identity Matrix (main diagonal = 1)

Custom Diagonal → np.diag()

Create diagonal from list

```
arr = np.diag([1,2,3])  
print(arr)
```

Output:

```
[[1 0 0]  
[0 2 0]  
[0 0 3]]
```

Extract diagonal from matrix

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])  
  
print(np.diag(arr))
```

Quick Revision

- `np.empty()` → fast, uninitialized array
- `np.eye(n)` → identity matrix
- `np.diag()` → create OR extract diagonal

How to Create NumPy Array with Random Numbers

1. Random Float (0 to 1) →

np.random.rand()

```
arr = np.random.rand(2,3)  
print(arr)
```

Range: 0 to 1

Use: sample data, testing

2. Random Integer →

np.random.randint()

```
arr = np.random.randint(0, 10, (2,3))  
print(arr)
```

Output:

```
[[2 4 5]  
 [0 4 6]]
```

Range: 0 to 9

Best for: real dataset simulation

3. Random Normal Distribution →

np.random.randn()

```
arr = np.random.randn(2,3)  
print(arr)
```

Output:

```
[[ 1.86074966  0.49561939 -0.18786008]  
 [ 1.40139861 -0.33736052  0.28313247]]
```

Mean = 0, Std = 1

Use: statistics, ML models

4. Random Choice →

np.random.choice()

```
arr = np.random.choice([10,20,30],  
size=(2,3))  
print(arr)
```

Output:

```
[[30 10 20]  
 [30 10 20]]
```

Picks random values from given list
Use: categorical data simulation

5. Set Seed (**IMPORTANT for Same Output**)

```
np.random.seed(42)
```

```
arr = np.random.rand(2,3)  
print(arr)
```

Output:

```
[[0.37454012 0.95071431 0.73199394]  
 [0.59865848 0.15601864 0.15599452]]
```

Same output every time

Used in:

- Testing
- Reproducibility

Data Type of NumPy Array (dtype)

1. Check Data Type

```
import numpy as np
```

```
arr = np.array([1, 2, 3])  
print(arr.dtype)
```

Output: int64 (system dependent)

◆ 2. Common Data Types (Must Know)

Type	Meaning
int32 , int64	Integer
float32 , float64	Decimal
bool	True/False
str / <U	String

3. Mixed Data → Auto Conversion

```
arr = np.array([1, 2.5, 3])  
print(arr.dtype)
```

Output: float64

Rule:

NumPy converts to highest precision type

4. Set Data Type Manually

```
arr = np.array([1,2,3], dtype=float)  
print(arr)
```

Output: [1. 2. 3.]

5. Convert Data Type → `astype()`

```
arr = np.array([1.5, 2.7, 3.2])
```

```
new_arr = arr.astype(int)  
print(new_arr)
```

Output: [1 2 3]

Decimal removed (not rounded)

6. Why `dtype` Matters (Real Use)

- Memory optimization
- Faster computation
- Required in ML models

What are Arithmetic Operations in NumPy

Arithmetic operations = element-wise mathematical operations on arrays

1. Addition (+)

```
a = np.array([1,2,3])  
b = np.array([4,5,6])
```

```
print(a + b)
```

Output: [5 7 9]

2. Subtraction (-)

```
a = np.array([1,2,3])  
b = np.array([4,5,6])  
  
print(a - b)
```

Output: [-3 -3 -3]

3. Multiplication (*)

```
a = np.array([1,2,3])  
b = np.array([4,5,6])  
  
print(a * b)
```

Output: [4 10 18]

4. Division (/)

```
a = np.array([1,2,3])  
b = np.array([4,5,6])  
  
print(a / b)
```

Output: [0.25 0.4 0.5]

5. Power (**)

```
a = np.array([1,2,3])  
b = np.array([4,5,6])  
  
print(a ** b)
```

Output: [1 4 9]

6. Scalar Operations (Single Value)

```
a = np.array([1,2,3])  
b = np.array([4,5,6])
```

```
print(a + 10)  
print(a + 10)
```

Output:

- [11 12 13]
- [2 4 6]

7. Key Concept (MOST IMPORTANT)

All operations are:

Element-wise (no loops needed)

Arithmetic Operations on 2D NumPy Array

1. Addition (+)

```
a = np.array([[1,2],[3,4]])
```

```
b = np.array([[5,6],[7,8]])
```

```
print(a + b)
```

Output:

```
[[ 6  8]  
 [10 12]]
```

Aggregation Functions in NumPy

Used to summarize data
(single value output)

1. Sum → `np.sum()`

```
arr = np.array([10,20,30,40])
```

```
print(np.sum(arr))
```

Output: 100

2. Mean (Average) → np.mean()

```
arr = np.array([10,20,30,40])
```

```
print(np.mean(arr))
```

Output: 25.0

3. Maximum → np.max()

```
arr = np.array([10,20,30,40])
```

```
print(np.max(arr))
```

Output: 40

4. Minimum → np.min()

```
arr = np.array([10,20,30,40])  
  
print(np.min(arr))
```

Output: 10

5. Standard Deviation → np.std()

```
arr = np.array([10,20,30,40])  
  
print(np.std(arr))
```

Spread of data

6. Variance → np.var()

```
arr = np.array([10,20,30,40])
```

```
print(np.var(arr))
```

Data variability

7. On 2D Array (VERY IMPORTANT)

```
arr = np.array([[1,2,3],[4,5,6]])
```

```
print(np.sum(arr))    # total
```

```
print(np.sum(arr, axis=0)) # column-wise
```

```
print(np.sum(arr, axis=1)) # row-wise
```

Output:

```
21
```

```
[5 7 9]
```

```
[ 6 15]
```

Shape and Reshaping in NumPy

1. Shape → arr.shape

Tells rows & columns

```
arr = np.array([[1,2,3],[4,5,6]])
```

```
print(arr.shape)
```

Output: (2, 3)

Meaning: 2 rows, 3 columns

2. Reshape → reshape()

Change array structure without changing data

```
arr = np.array([1,2,3,4,5,6])
```

```
new_arr = arr.reshape(2,3)
```

```
print(new_arr)
```

Output:

```
[[1 2 3]
```

```
[4 5 6]]
```

3. Important Rule

Total elements must match

6 elements $\rightarrow$ (2 $\times$ 3)

6 elements $\rightarrow$ (3 $\times$ 2)

6 elements $\rightarrow$ (4 $\times$ 2)

4. Auto Reshape → -1 (Very Useful)

```
arr = np.array([1,2,3,4,5,6])
```

```
new_arr = arr.reshape(2, -1)  
print(new_arr)
```

Output:

```
[[1 2 3]  
 [4 5 6]]
```

NumPy automatically calculates column

Explanation:

- Total elements = 6
- Rows = 2
- Columns = automatically = 3

5. Convert 2D → 1D → reshape(-1)

```
arr = np.array([[1,2,3],[4,5,6]])
```

```
flat = arr.reshape(-1)  
print(flat)
```

Output: [1 2 3 4 5 6]

ndim in NumPy

ndim = number of dimensions (axes)
in array

1. 1D Array

```
arr = np.array([1,2,3])
```

```
print(arr.ndim)
```

Output: 1

Broadcasting in NumPy

Definition (Perfect Line):

Broadcasting is the ability of NumPy to perform operations on arrays of different shapes by automatically expanding the smaller array.

Core Rule (MOST IMPORTANT)

Shapes compare from right → left

Two dimensions are compatible if:

- Equal
- OR one of them is 1

Example 1 (Scalar Broadcasting)

```
a = np.array([1,2,3])  
print(a + 10)
```

Internally:

```
[1 2 3]  
[10 10 10]
```

Output: [11 12 13]

Example 2 (1D → 2D Broadcasting)

```
a = np.array([[1,2,3], [4,5,6]])
```

```
b = np.array([10,20,30])
```

```
print(a + b)
```

Internally:

[10 20 30]

[10 20 30]

Output:

[[11 22 33]

[14 25 36]]

Why Broadcasting is Powerful

- No loops → fast execution
- Clean code
- Handles large datasets easily

Indexing and Slicing in NumPy

1. Indexing (Access Single Value)

1D Array

```
arr = np.array([10,20,30,40])
```

```
print(arr[0]) # first element
```

```
print(arr[-1]) # last element
```

Output: 10, 40

2D Array

```
arr = np.array([[1,2,3], [4,5,6]])
```

```
print(arr[0,1]) # row 0, col 1
```

Output: 2

2. Slicing (Access Multiple Values)

1D Array

```
arr = np.array([10,20,30,40,50])
```

```
print(arr[1:4])
```

Output: [20 30 40]

Rule:

start : end (end excluded)

2D Array

```
arr = np.array([[1,2,3],  
[4,5,6]])
```

```
print(arr[:, 1]) # all rows, column 1
```

Output: [2 5]

Row Slicing

```
print(arr[1, :])
```

Output: [4 5 6]

Sub-matrix

```
print(arr[0:2, 1:3])
```

Output:

```
[[2 3]  
 [5 6]]
```

3. Step Slicing

```
arr = np.array([10,20,30,40,50])
```

```
print(arr[::2])
```

Output: [10 30 50]

Iterating NumPy Arrays

Iteration = array ke elements ko ek-ek karke access karna

1. Iterate 1D Array

```
arr = np.array([10,20,30])
```

```
for i in arr:  
    print(i)
```

Output:

```
10  
20  
30
```

2. Iterate 2D Array (Row-wise)

```
arr = np.array([[1,2,3], [4,5,6]])
```

```
for row in arr:  
    print(row)
```

Output:

```
[1 2 3]
```

```
[4 5 6]
```

3. Iterate Each Element (Nested Loop)

```
arr = np.array([[1,2,3], [4,5,6]])
```

```
for row in arr:  
    for val in row:  
        print(val)
```

Output: 1 2 3 4 5 6

4. Using `nditer()` (Best Method)

```
arr = np.array([[1,2,3],[4,5,6]])
```

```
for i in np.nditer(arr):  
    print(i)
```

Output: 1 2 3 4 5 6

Works for:

- Any dimension
- Clean & efficient

Work on = 1D , 2D ,3D

5. Iterate with Index → ndenumerate()

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
```

```
for index, value in np.ndenumerate(arr):  
    print(index, value)
```

Output:

```
(0, 0) 1  
(0, 1) 2  
(0, 2) 3  
(1, 0) 4  
(1, 1) 5  
(1, 2) 6  
(2, 0) 7  
(2, 1) 8  
(2, 2) 9
```

Copy vs View in NumPy

1. Copy (Independent Data)

```
arr = np.array([1,2,3])  
copy_arr = arr.copy()
```

```
copy_arr[0] = 100
```

```
print(arr)  
print(copy_arr)
```

Output:

```
[1 2 3]  
[100 2 3]
```

Key Point:

Original array change nahi hota

2. View (Same Memory)

```
arr = np.array([1,2,3])  
view_arr = arr.view()
```

```
view_arr[0] = 100
```

```
print(arr)  
print(view_arr)
```

Output:

```
[100 2 3]  
[100 2 3]
```

Key Point:

Original array bhi change ho jata hai

Core Difference

Feature	Copy	View
Memory	New	Same
Change effect	No	Yes
Safe		

Real Use Insight:

- Use `copy()` → when safety needed
- Use `view()` → when performance needed

NumPy Join & Split Arrays

Used to:

- Join → combine multiple arrays
- Split → break one array into parts

PART 1: Join Arrays

1. `np.concatenate()` (Most Used)

```
a = np.array([1,2,3])
```

```
b = np.array([4,5,6])
```

```
result = np.concatenate((a,b))
```

```
print(result)
```

Output: [1 2 3 4 5 6]

2. Join 2D Arrays (axis important)

```
a = np.array([[1,2],[3,4]])
```

```
b = np.array([[5,6],[7,8]])
```

```
result = np.concatenate((a,b), axis=0)
```

```
print(result) # row-wise
```

Output:

```
[[1 2]
```

```
[3 4]
```

```
[5 6]
```

```
[7 8]]
```

```
print(np.concatenate((a,b), axis=1))
```

```
# column-wise
```

Output:

```
[[1 2 5 6]
```

```
[3 4 7 8]]
```

3. np.stack() (Adds new dimension)

```
print(np.stack((a,b), axis=0))
```

Creates extra dimension

PART 2: Split Arrays

1. np.split()

```
arr = np.array([1,2,3,4,5,6])
```

```
result = np.split(arr, 3)  
print(result)
```

Output:

```
[array([1,2]), array([3,4]), array([5,6])]
```

2. Split 2D Array

```
arr = np.array([[1,2],[3,4],[5,6]])
```

```
result = np.array_split(arr, 3)  
print(result)
```

Output:

```
[array([[1, 2]]), array([[3, 4]]), array([[5, 6]])]
```

Important Rule

- concatenate() → shapes must match
- split() → equal division required
- array_split() → flexible split

NumPy Array Functions

These are built-in functions to manipulate arrays

1. np.sort() → Sorting

```
arr = np.array([3,1,2,5,6,8,29,])
```

```
print(np.sort(arr))
```

Output: [1 2 3 5 6 8 29]

2. np.where() → Condition Check

```
arr = np.array([10,20,30,40])  
where = np.where(arr > 20)  
  
print(where)
```

Output: (array([2, 3]),)

Returns index

3. np.unique() → Unique Values

```
arr = np.array([1,2,2,3,3,3,3])  
unique = np.unique(arr,  
return_counts=True)  
  
print(unique)
```

Output:
(array([1, 2, 3]), array([1, 2, 4]))

4. np.reshape() → Change Shape

```
arr = np.array([1,2,3,4,5,6])  
reshape = np.reshape(arr, (2,3))  
  
print(reshape)
```

Output:

```
[[1 2 3]  
 [4 5 6]]
```

5. np.flatten() → Convert to 1D

```
arr = np.array([[1,2],[3,4],[9,8]])  
flatten = arr.flatten()  
  
print(flatten)
```

Output: [1 2 3 4 9 8]

6. np.transpose() → Swap Rows & Columns

```
arr = np.array([[1,2],[3,4],[5,6]])  
trn = np.transpose(arr)  
  
print(trn)
```

```
[[1 3 5]  
 [2 4 6]]
```

7. np.clip() → Limit Values

```
arr = np.array([10,20,30,40])
```

```
print(np.clip(arr, 15, 35))
```

Output: [15 20 30 35]

Shuffle → np.random.shuffle()

Array ko random order me arrange karta hai

```
arr = np.array([1,2,3,4,5])
```

```
np.random.shuffle(arr)
```

```
print(arr)
```

Output: random order (same array change hota hai)

Original array modify hota hai (in-place)

Ravel → np.ravel()

Multi-dimensional → 1D array

```
arr = np.array([[1,2],[3,4]])
```

```
flat = np.ravel(arr)  
print(flat)
```

Output: [1 2 3 4]

Note:

- Returns view (not copy) in most cases

Insert & Delete Functions in NumPy

1. Insert → np.insert()

Array me new value add karna (specific position par)

```
arr = np.array([10,20,30])
```

```
new_arr = np.insert(arr, 1, 99)  
print(new_arr)
```

Output: [10 99 20 30]

2D Array Insert

```
arr = np.array([[1,2],[3,4]])
```

```
# column insert
```

```
new_arr = np.insert(arr, 1, [9,9], axis=1)
```

```
print(new_arr)
```

```
[[1 9 2]
```

```
[3 9 4]]
```

2. Delete → np.delete()

Array se value remove karna

```
arr = np.array([10,20,30,40])
```

```
new_arr = np.delete(arr, 2)
```

```
print(new_arr)
```

Output: [10 20 40]

2D Array Delete

```
arr = np.array([[1,2],[3,4]])
```

```
# row delete
```

```
new_arr = np.delete(arr, 0, axis=0)
```

```
print(new_arr)
```

Output:

```
[[3 4]]
```

Important Point

Both functions:

Original array ko change nahi karte
(new array return hota hai)

Matrix in NumPy

Matrix = 2D Array (rows × columns)

Used for:

- Data tables
- Calculations
- Machine Learning

1. Create Matrix (Standard Way)

```
A = np.array([[1,2,3],  
              [4,5,6]])
```

Shape = (2,3) → 2 rows, 3 columns

2. Types of Matrix (Important)

1. Row Matrix

```
np.array([[1,2,3]])
```

2. Column Matrix

```
np.array([[1],  
          [2],  
          [3]])
```

3. Zero Matrix

```
np.zeros((2,2))
```

4. Identity Matrix

`np.eye(3)`

3. Matrix Operations

Addition

$A + B$

Subtraction

$A - B$

Element-wise Multiplication

$A * B$

4. Transpose → .T

```
A = np.array([[1,2,3],  
              [4,5,6],[11,12,13]])
```

```
print(A.T)
```

Output:

```
[[ 1  4 11]  
 [ 2  5 12]  
 [ 3  6 13]]
```

5. Indexing in Matrix

`A[0,1]` # element

`A[0,:]` # row

`A[:,1]` # column

```
array([ 2,  5, 12])
```

7. Important Math Functions

```
np.sum(A)
```

```
np.mean(A)
```

```
np.max(A)
```

8. Real Use Case

Example: Sales Data

```
sales = np.array([[100,200,300],  
                  [150,250,350]])
```

```
print(np.sum(sales, axis=1)) # total per row
```

Output: axis = 0

```
[250 450 650]
```

Output: axis = 1

```
[600 750]
```

ndim in NumPy

ndim = number of dimensions (axes)
in array

Example 1 (1D)

```
arr = np.array([1,2,3])  
print(arr.ndim)
```

Output: 1

Example 2 (2D)

```
arr = np.array([[1,2],[3,4]])  
print(arr.ndim)
```

Output: 2

Example 3 (3D)

```
arr = np.array([[[1,2],[3,4]]])  
print(arr.ndim)
```

Output: 3

Key Point

- 1D → line of data
- 2D → table (rows + columns)
- 3D → multiple tables